SheetShrink: Efficient Spreadsheet Cleanup

Alexander Aue-Johr
Institute of Rural Studies
Thünen-Institute
Braunschweig, Germany
Harz University of Applied Sciences
Doctoral Candidate, Germany
ORCID 0009-0001-8683-3630

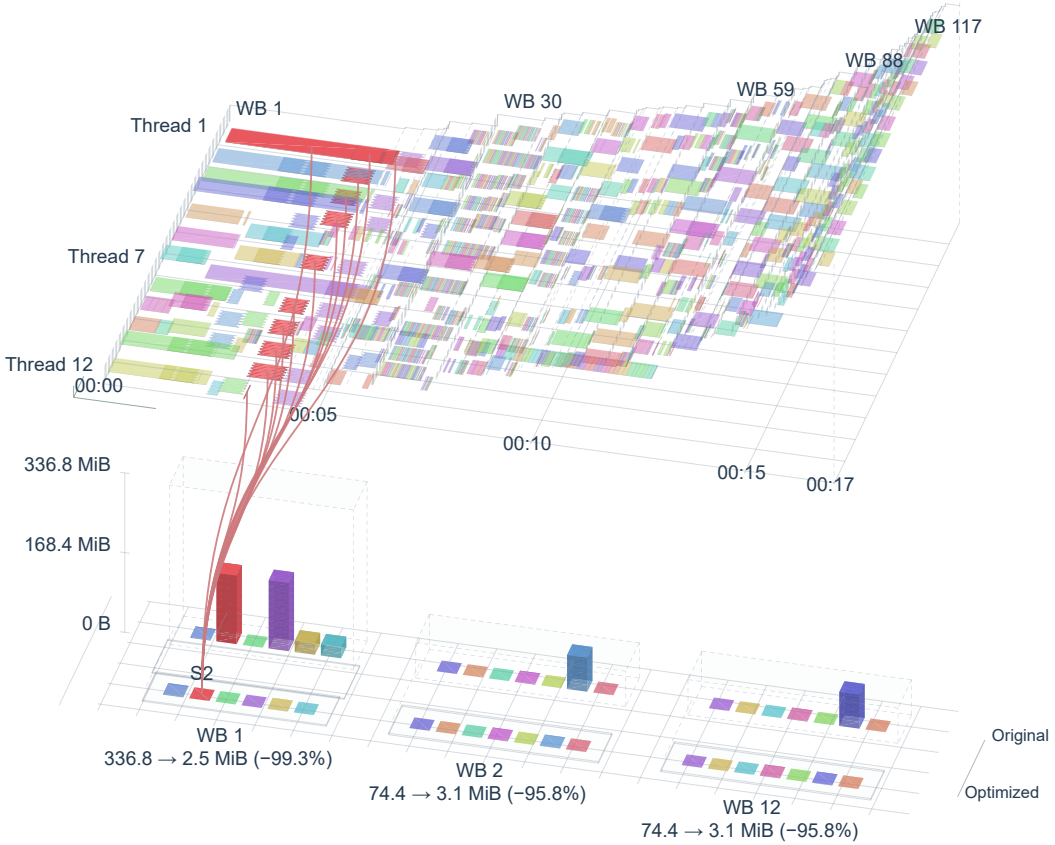


Fig. 1. Recorded development run on 144 Enron workbooks selected for large absolute XLSX reductions, using 12 workers. Work intervals follow workbook admission order (WB denotes workbook, numbered by admission). Thread labels identify worker lanes. The lower view compares the three largest uncompressed packages on one linear byte scale. Curves connect worksheet S2 in WB 1 to its work items. Observed wall time was 17.077 s.

***Abstract*—Empty and redundant cell records can make XLSX files expensive to store and load. SheetShrink removes redundant cell and formatting records directly from XLSX packages before they are opened in Excel. Its removal rules account for inherited formats, automatic row heights, and references from formulas and charts. A Rust implementation scans worksheet XML as bytes and processes worksheets in parallel. We evaluate cleanup on 1,182 Destatis, 8,623 Data.gov, and 30,301 prepared Enron workbooks. Aggregate compressed file sizes decrease by 15.48%, 5.29%, and 18.92%, respectively. A further 494 Data.gov files use unsupported XML namespaces and are skipped. Opening and saving experiments measure how cleanup changes processing time and memory use in spreadsheet libraries and Excel. Native Excel comparisons and controlled regression cases examine changes in appearance, including empty cells whose fonts or borders affect the rendered worksheet.**

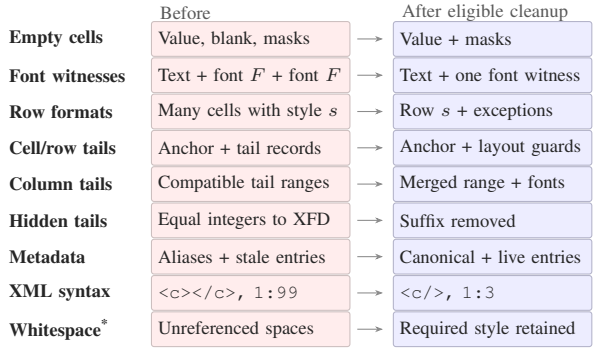


Fig. 2. Cleanup rules with layout, protection and dependency checks. Cell cleanup accounts for inherited fills and borders. Row promotion requires compatible fixed heights. Hidden constant tails require at least 4,096 cells ending at XFD. *Whitespace removal is optional. Required witnesses and coordinates remain.

I. Introduction

An XLSX workbook can contain a small table alongside many empty cell records and redundant row and column records. Decompressing and parsing these records adds work. Removing every empty-looking record is unsafe. An empty cell can preserve a border, override an inherited fill, or influence automatic row height through its font. A whitespace value can prevent text from extending into its neighbour. Cleanup must account for how stored records affect the workbook.

The Destatis migration workbook `wanderungen-2010120217005.xlsx` illustrates the problem. In `Tabelle 3.4`, 21 rows contain zeros from column L to Excel's last column, XFD, totaling 343,833 values. Removing this hidden content requires explicit removal rules and dependency checks. Section III-E explains the rule and which values it retains.

We designed SheetShrink for use without Windows or Excel, including Python and R workflows. Direct package processing also avoids Excel's opening cost, which was high in our experiments.

Cleanup decisions use stored values, styles and references. SheetShrink retains compact records needed to preserve required workbook behavior. Excel rendering and calculated row heights are used later to validate the output.

The output remains an editable XLSX workbook and must preserve required data, dependencies and appearance. File-size reduction alone is insufficient.

Figure 2 summarizes cleanup of cells, rows, columns and workbook metadata. Existing styles and representative records preserve required effects. Saving in Excel can further compact style and shared-string tables.

We aim to make cleanup followed by opening faster than opening the original bloated workbook. Single-thread comparisons isolate improvements in the work performed. Parallel corpus experiments measure throughput and memory use.

II. Preservation and Comparison Contract

A. Observable spreadsheet behavior

Cell records can contain formatting, protection, references and values that affect other cells. SheetShrink removes redundant records while retaining those required by its preservation contract. This paper evaluates the production cleanup policy.

The relevant visual area includes cells with meaningful values or formulas, visible fills or borders, and graphical objects. Row and column formats, conditional formatting and overflowing text can also affect it. Validation covers the union of the original and transformed areas to include removed content. The contract permits layout changes in empty trailing rows or columns. Changes that move or alter relevant content remain subject to validation.

B. Information available before native loading

Cleanup uses stored values, formulas, styles, references and layout attributes to establish which records can be removed before Excel renders the workbook. Native row heights and before/after images are available only in the separate validation workflow.

An Excel-hosted cleaner could use these observations to guide removal, for example by preserving a measured row height with *customHeight* enabled [1]. It would require Excel and incur its opening cost, including in Python and R workflows.

C. Removal decisions and compact preservation

SheetShrink retains a representative record when it preserves a required effect. Examples include row-font sentinels and reuse of an existing cell style as a row format. The checks account for inherited formatting, gaps, merges, protection and available styles. Unresolved cases retain the required original records.

The production policy removes hidden constant tails only after dependency and layout checks [2]. Ordinary nonempty hidden tables remain intact, including unreferenced values. A missing reference alone is insufficient grounds for deleting hidden data.

D. Validation and attribution boundaries

DeltaSieve compares selected regions after independently rebuilding both workbooks in the same Excel environment [3]. Save and rebuild controls help identify changes introduced by Excel.

For XLS sources, conversion precedes the XLSX cleanup subject and has its own failure accounting. A file requiring repair or inaccessible protected content receives an explicit unsuccessful outcome.

Matching images establish equality within the captured regions. We evaluated region selection using independent native reference captures and deliberately modified test workbooks [4], [5]. We retain pixel differences and investigate their causes through repeated rendering, volatile dependencies and capture geometry.

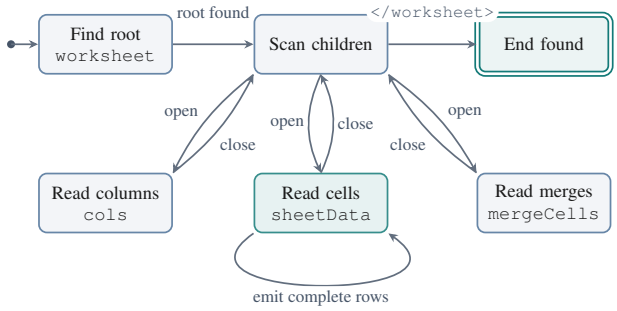


Fig. 3. Worksheet-stream states. Child readers return to the dispatcher; read boundaries preserve state and partial input. Reconstruction requires a completed worksheet; incomplete input, I/O failures and unsupported XML return an error.

III. Method

A. Representation and analysis stages

SheetShrink removes redundant records from an XLSX ZIP package while preserving workbook structure and cell coordinates. Before evaluating individual cells, it reads styles, shared strings, relationships, merged regions and dependency sources. A style index or empty value alone cannot establish redundancy.

Worksheet analysis reads decompressed bytes without creating an XML object for each cell. A state machine distinguishes headers, columns, sheet data, merges and the remaining XML. Tag and attribute readers track nesting and quotation marks, preventing coordinate-like text in attribute values from being interpreted as cell addresses. Partial markers are retained across buffer boundaries. Metadata is retained when later decisions need it. Names, references and other interpreted values may require decoding.

Memory use grows with worksheet size. SheetShrink retains sheet-data bytes, analysis context and candidate metrics for later passes. Decompression, rewriting and compression also contribute to peak memory. The supported format is unprefixed Transitional SpreadsheetML. Unsupported namespaces, encodings and ambiguous structures trigger preservation or exclusion [2]. Bounded package-header checks and declarations encountered during streaming identify unsupported representations before opening in Excel.

Figure 3 shows how the worksheet parser dispatches column, cell and merge readers. It retains partial state across buffer boundaries and processes complete rows while retaining an unfinished row. Reconstruction requires the worksheet end.

B. Parser correctness and regression controls

The byte scanner must recognize XML structure and track parser state. It must check element boundaries, quoted attributes, empty elements and incomplete input. Established SAX and DOM parsers provide extensively tested syntax handling. Memory-safe code can still misclassify content and produce an incorrect workbook.

A Data.gov workbook exposed a parsing error. Its visible strings used `<v xml:space="preserve">`. The scanner recognized only `<v>`, misclassified these cells as empty and removed a heading and other strings. It now accepts valid element-name delimiters and retains value and inline-string elements with attributes. Tests reject lookalike names such as `<value>` [6]. Another regression test checks that replacing a self-closing defined-name element leaves the following paired element intact [7].

SheetShrink publishes output only after parsing reaches the worksheet end. An incomplete parse raises an error, deletes partial output and leaves any existing destination unchanged. Integration tests cover failure after another worksheet has been processed [8]. Small fixtures test comments, CDATA, quotes, nesting limits and source offsets at every input chunk size.

At the time of writing, 31 tests cover XML parsing, namespaces, shared strings, ranges, defined names and streaming readers. Another 155 tests cover cell, row and tail preservation, including inherited formatting and content boundaries. Further tests cover dependencies, styles, output construction and execution. Each test function counts once, including parameterized tests. The artifact documents the revision, test paths and counting criteria.

C. Effects and compact preservation witnesses

Numeric values, formulas and inline strings count as content, including zero and shared-formula followers without their own formula text. Empty-cell removal depends on cell, row and column formatting. A no-fill or borderless cell style must remain if removal would expose an inherited fill or border. Without an inherited effect to suppress, it can be redundant. Style components and their application flags are checked together.

An empty cell's font can set automatic row height and shift later content. SheetShrink keeps a required representative for repeated font records. Equivalent fonts can share an identity despite different style indices. Column-font masks and merged spans also constrain removal. A record elsewhere in the row may not preserve a required local mask. Figure 4 isolates the mechanism in a constructed native Excel control. One witness preserves the observed height. Removing both moves the next visible row.

Even with a fixed row height, an empty cell's font can affect text alignment within the row. SheetShrink therefore distinguishes empty rows from rows containing text. Protection does not remove this font effect. Within the retained data area, protection formats also preserve editing restrictions. Separate checks govern removal beyond that area.

Some empty records preserve layout relationships. Center-across-selection needs the consecutive blank cells after the text cell. A borderless row can suppress column borders, while an empty row's height can set their vertical extent. Column widths can support visible row borders. Charts can inherit number formats from empty first series cells. Required records can therefore extend beyond the last nonempty value. Figure 5 shows an Enron counterexample in which Check

Source

```
<row r="2">
 <c r="A2">...</c>
 <c r="B2" s="1"/>
- <c r="C2" s="1"/>
</row>
```

Guarded result

```
<row r="2">
 <c r="A2">...</c>
 <c r="B2" s="1"/>
</row>
```

Original

Font witness control			
10 pt text			
Next visible row			

One witness

Font witness control			
10 pt text			
Next visible row			

No witness

Font witness control			
10 pt text			
Next visible row			

Fig. 4. Font-witness control. A2 uses Arial 10; empty B2 and C2 use Arial 18. Retaining either blank preserves the rebuilt rendering. Removing both reduces row 2 from 23.25 to 12.75 points and shifts the following row. XML is abbreviated; images are native Excel print exports.

Performance's smaller output exposes a different inherited fill. The observation concerns the pictured rebuilt region.

D. Guarded promotion and tail pruning

SheetShrink can replace repeated cell styles with an existing cell style applied to the row. The row must be visible, unprotected and have no row style. A stored height and enabled `customHeight` flag are required. The `ht` attribute alone permits automatic sizing. Content, merges, gaps and inherited column formats must remain compatible with the row format. Figure 6 illustrates the full-width case [9].

The most frequent empty-cell style must occur at least twice. SheetShrink counts all styles before checking eligibility, preventing an eligible minority from replacing an ineligible majority. Ties favor the smaller existing style index. Dense promotion requires at least 1,024 occurrences. The bounded-gap route allows up to 64 missing columns and restores required original formats after inheritance checks. Sparse promotion requires equivalent inheritance at every gap and excludes merged rows. Uniform-fill promotion also checks fills on content cells. Unknown metadata or style identities block promotion.

Pruning removes formatting-only records beyond the last required content or layout anchor. Font-only blanks beyond this boundary have no later content to shift. Fills, borders, conditional formatting and drawings can extend the boundary. A white row fill is redundant only if compatible column fills cover every column through XFD. Compatible column ranges can be merged while preserving local overrides and geometry needed for borders.

E. Hidden content and the Wanderungen case

SheetShrink uses a conservative heuristic to remove certain hidden numeric tails. A candidate must contain at least 4,096 identical integers and extend to XFD. This restriction targets large repeated tails such as those in Wanderungen. Candidates must also pass cell-level layout and dependency checks. Gaps, formulas, text, differing values and unsupported numeric forms reject the band. Source cells contribute before style cleanup. The common zero path compares value bytes directly, while one admitted row span replaces per-cell entries in the later reference map. Temporary row-local candidates still exist.

In Wanderungen, `Tabelle 3.4` contains 21 zero tails in rows 11–30 and 32, from L through XFD. Each contains 16,373 zeros, totaling 343,833. ZIP inspection confirms the counts and records the source hash. The cleanup investigation documented 376,573 removed zeros across two worksheets, 243 preserved nonzero values and a retained 237-zero tail [10]. This structural rule can also match a deliberate full-width constant matrix. Repetition alone cannot establish user intent.

F. Workbook dependencies and deferred rewriting

SheetShrink postpones reference-sensitive deletions until it has checked supported formulas, names, charts, objects and pivot dependencies across the workbook. This includes references from later worksheets. Workbook relationships and sheet scope identify reference targets. Shared-formula followers shift relative axes and preserve absolute axes. The implementation protects enclosing referenced ranges without enumerating each shared-formula follower's references. Dynamic or unresolved references and unsupported expressions block deletion [11].

A reference overlapping a candidate hidden span protects the entire span. Rewriting removes only candidates that pass all checks. A removed value's format remains if needed to suppress inherited formatting. Affected rows are cleaned again after obsolete tail anchors are removed, allowing records kept solely for those anchors to disappear. Default preserves whitespace because it can control text overflow and row layout.

Metadata cleanup follows separate rules. Calculation-chain cleanup identifies removed cells by worksheet and preserves entries whose targets cannot be resolved. SheetShrink does not evaluate formulas or generate cached results. Built-in alias cleanup matches names and local scope without checking whether their formulas are equivalent.

G. Parallel work and output ownership

Figures 7 and 8 show the task hierarchy and a schematic execution across threads. Worksheet identity remains shared across its chunks, while ordered assembly separates execution order from output order. Workbook, worksheet and row-chunk tasks share one execution budget. Each active workbook holds a permit until its output is finalized. Workers

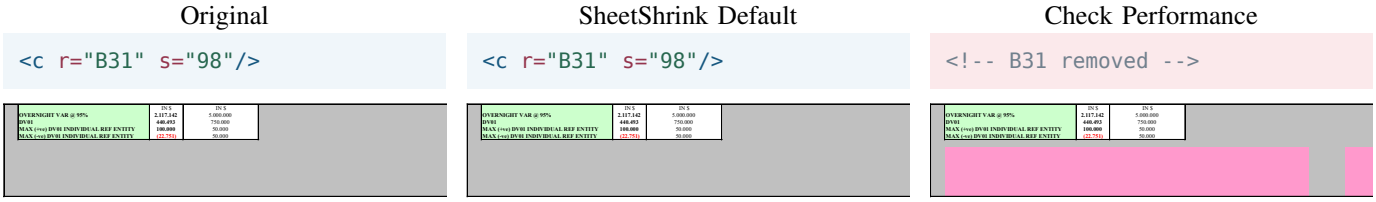


Fig. 5. An empty cell masks inherited formatting. In the Enron example, gray B31 overrides a pink column fill. SheetShrink preserves the mask; Check Performance removes it and exposes pink. Native print exports match for Original and SheetShrink after full rebuild within the displayed region. XML is abbreviated.



Fig. 6. Guarded row-style promotion replaces 16,383 repeated empty-cell records with an existing row format while retaining the value cell. Red marks removals and green additions. Intermediate cells are omitted. Promotion requires fixed-height, protection and inheritance checks described in the method. Style indices are local.

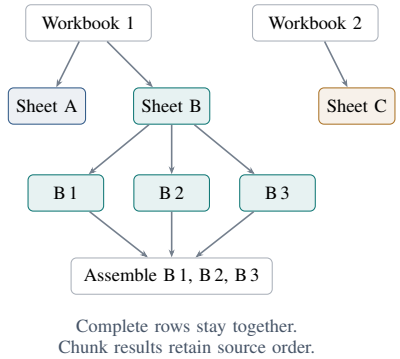


Fig. 7. Task hierarchy. Workbook admission bounds active inputs; sheets and row-aligned chunks share the Rayon pool. B 1–B 3 belong to one worksheet and are assembled in source order. Colors match Figure 8.

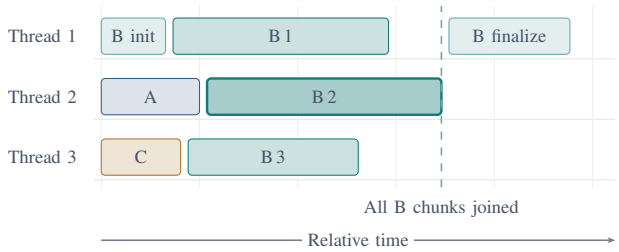


Fig. 8. One possible execution of Figure 7. Chunks from one sheet can run on different threads before ordered assembly. Bar lengths are schematic; individual steal events are not recorded.

can process other workbooks while a long worksheet runs. Large worksheets are split into row-aligned chunks for Rayon workers, keeping each row's font, promotion and tail decisions together. The planner estimates work in bytes and adjusts chunk boundaries to complete rows. Small ranges run serially. Each chunk has separate metrics, and results are assembled in source order.

Worksheet admission accounts for ready chunks. Smaller worksheets may wait while larger chunk tasks are pending. Batches limit the number of retained chunk results, while the assembled output continues to grow. Workbook contexts, output buffers and deferred candidates also consume memory. Thread, active-workbook and writer counts are separate experimental parameters.

Workers compress changed worksheet members in parallel. One writer updates each workbook's ZIP archive and publishes the completed temporary file after successful processing. Unchanged members can reuse their compressed bytes. If every member is unchanged, SheetShrink copies the original file byte for byte. A fast identity check uses CRC32 and uncompressed size.

SheetShrink preserves existing style and shared-string indices. This avoids rewriting unaffected ZIP members, whose compressed bytes can be copied directly. Further reduction through global table compaction or saving in Excel is outside the cleanup evaluated here.

IV. Evaluation

We evaluate three questions. RQ1 measures compressed-file reduction and the records retained for preservation. RQ2 measures cleanup time, memory and total cleanup-plus-opening time. RQ3 tests structural preservation and Excel-rendered appearance within the stated validation scope.

A. Corpus identity and outcome accounting

Data.gov, Destatis and Enron are evaluated separately. Archive checksums, member names and duplicate occurrences identify each corpus version. XLS conversion is reported separately from XLSX cleanup. Recursive extraction of Internet Archive's EDRM Enron v2 XML archives inspected nested archives and embedded containers. It yielded 34,065 distinct SHA-256-addressed candidates, including legacy Excel files and other OLE objects. The prepared Enron XLSX evaluation population is described below.

For native visual stress tests, we select the 49 workbooks with the largest absolute compressed-file reduction from each corpus, totaling 147 original identities. Every processor baseline uses these identities. Preparation, cleanup and visual comparison have separate outcome counts.

The frozen XLSX inventories contain 9,117 Data.gov members totaling 25.273 GB and 1,182 Destatis members totaling 2.364 GB, measured as compressed file bytes in decimal GB. Each member has a SHA-256 identity. Outcomes distinguish completed comparisons, differences, failures, unsupported formats and unfinished work. Retries are counted separately. Identical packages can bypass visual comparison. Failed captures cannot establish equality.

B. Cleanup screening outcomes

Fresh release-build runs produced 1,182 Destatis, 8,623 Data.gov, and 30,301 prepared Enron XLSX outputs. Across those outputs, aggregate compressed file bytes fell by 15.48%, 5.27%, and 18.87%, respectively. The smaller, equal-size, and larger output counts were 1,045/30/107 for Destatis, 4,755/3,522/346 for Data.gov, and 21,565/8,415/321 for Enron. Hash checks confirmed byte identity for all equal-size outputs in these runs. Data.gov also had 494 unsupported-namespace skips. With sixteen workbook slots and sixteen Rayon threads, elapsed times were 119.7, 1,134.5, and 181.6 s, respectively, including analysis statistics and full replay events alongside concurrent native validation [12]. All three runs reached completion with zero reported processing failures. The host used an AMD Ryzen 7 7800X3D with eight physical cores, sixteen logical processors, 95.1 GiB OS-visible RAM and Windows 11 Home build 26200. Interactive replays accompany the Destatis, Data.gov and prepared Enron XLSX runs; replay clocks additionally include final reporting. Successful processing of these inputs does not establish correctness for every valid serialization. These descriptive results include files that grew and use only processed input bytes as the denominator. Enron's denominator begins after preparation. Visual preservation is evaluated separately. Figure 9 shows the workbook and worksheet distributions for these runs. Empty-cell cleanup and trailing layout rows account for prominent worksheet XML reductions. The contribution of column range merging is especially visible in Destatis. The worksheet panels exclude 88 prepared Enron workbooks without worksheet XML observations. Workbook payload sizes use all uncompressed ZIP member sizes. Among 9,117 Data.gov inputs, 493 workbooks contained explicitly prefixed worksheet data elements. After experimental namespace normalization and Default cleanup, only two produced smaller compressed packages, by 6.415% and 0.047% relative to the normalized baseline. Cell-record removal in other outputs did not reduce package size [13].

Corpus testing exposed parsing and preservation defects. Enron controls identified missing center-across-selection spans and fill masks by restoring the affected original cell records. These findings informed regression fixtures and native Excel controls that complement the screening results [14].

C. Sampling and reference controls

We select the random sample from a frozen eligible population using a recorded seed before inspecting visual results. Sampling strata reflect corpus, package size and structural features known before validation. Low-reduction and unchanged files remain included. A separately labelled development stress set contains large and historically problematic workbooks.

Where feasible, sampled files receive full native reference captures of the relevant visual area, independently of the candidate selector. Timeouts remain in the denominator, without substituting easier workbooks. Test mutations introduce known visible changes to values, styles, geometry, conditional dependencies and objects. Changes normalized away by Excel are reported separately.

D. Performance and comparison subjects

We compared direct open–save with fresh six-worker SheetShrink cleanup followed by the same library's open–save. Across two laptop workbooks, all 33 cleanup-assisted trials completed under a 4 GiB commit limit, compared with 21 direct trials. Median time improved in all seven fully completed pairs. Migration with openpyxl took 111.3 s directly and 9.7 s including cleanup.

Figures 10 and 11 show payload reduction and subsequent processing for selected cases. The artifact retains full library matrices, absolute payloads and per-trial phases. The laptop experiment contains 288 attempts, with eighteen additional Enron attempts on the larger-memory host. Separate SheetShrink controls match the native candidates' member manifests. Full per-trial member inventories are unavailable for timed fresh outputs.

Figure 11 reports library open–save time and peak working-set memory. Times are medians of completed attempts. Memory peaks are maxima across all three attempts. Stopped attempts retain their elapsed time and memory boundary. Cleanup time and coordinator memory are excluded. The same library processes fresh SheetShrink outputs and prepared external-tool outputs. The artifact also retains pipeline working sets, job-commit peaks and fresh cleanup-plus-processing totals.

The host is a Ryzen 5 4500U laptop with six logical processors, 8 GiB RAM and Windows 11. Each job has a 4 GiB commit limit and 900 s timeout, without a free-RAM reserve. SheetShrink revision `765c480` uses six Rayon and six writer workers. Native visual comparisons independently rebuild both workbooks before capture.

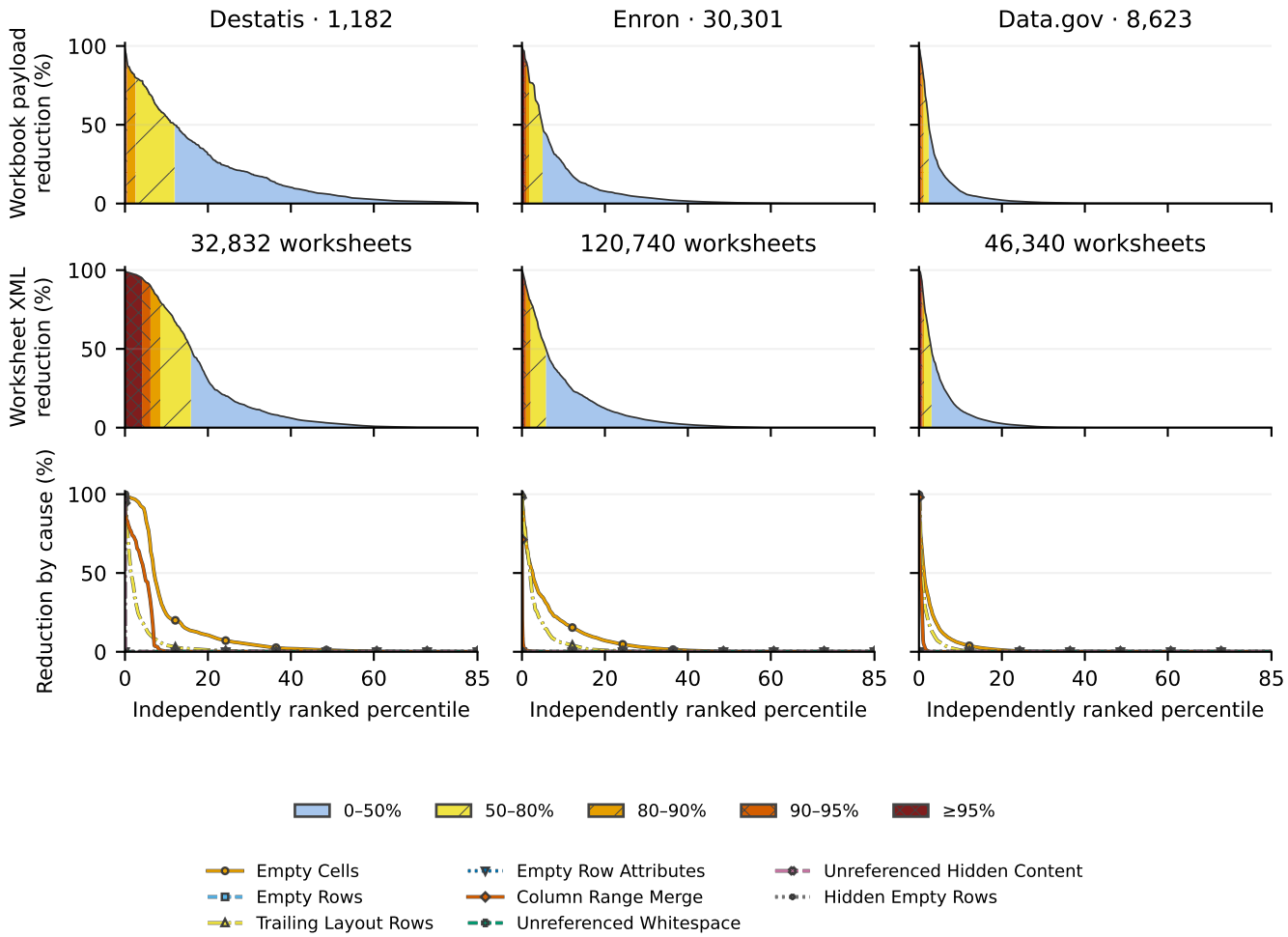


Fig. 9. Uncompressed-size reductions across the processed corpora. Rows show complete workbook payload, worksheet XML and per-cause worksheet XML reductions. Curves independently rank observations by decreasing reduction. The display ends at the 85th percentile; remaining positive reductions are below 0.5 percentage points. Colors and hatching identify reduction tiers. Data.gov namespace skips are excluded. Enron starts from prepared XLSX files; worksheet panels exclude 88 workbooks without worksheet XML.

E. Larger-memory-host follow-up

We repeated the four workbook/library combinations that reached the laptop's 4 GiB commit limit on an AMD Ryzen 7 7800X3D with sixteen logical processors and 95.1 GiB of usable RAM. Each combination received three Original and three fresh SheetShrink trials at both 4 GiB and 24 GiB process-tree commit limits, totaling 48 trials. The same SheetShrink binary uses sixteen Rayon and sixteen writer workers, with one active workbook. Trials reserve 8 GiB of physical RAM and commit headroom and time out after 900 seconds. Software versions are Python 3.12.14, openpyxl 3.1.5, pandas 3.0.1, R 4.6.2, readxl 1.6.0 and writexl 2.0.1 [15].

At 4 GiB, all twelve Original trials reached the memory limit, while all twelve fresh-cleanup pipelines completed. At 24 GiB, all six Original trials for Migration/readxl and QT/RR/openpyxl completed, while Migration's six pandas trials reached the limit. All twelve fresh-cleanup pipelines completed. The artifact retains completed and memory-limited results. Trials stopped for memory receive no runtime ratio. For Migration/readxl and QT/RR/openpyxl, cleanup plus processing was 2.97 and 24.28 times faster, respectively, taking 131.953 versus 392.149 s and 11.519 versus 279.638 s.

The higher memory limit lets some previously memory-limited trials complete, allowing runtime comparisons. Native visual comparisons, another cleanup benchmark and read-only evidence analysis ran concurrently, with filesystem caches retained. The ratios describe paired results under these conditions. Sampled physical RAM and commit headroom stayed above the declared reserve throughout all 48 trials.

After timing, regenerated controls for both inputs matched the native candidates' uncompressed member names, lengths and SHA-256 hashes. Migration uses a documented normalized baseline. QT/RR's selected capture plan contains no captures. Timed fresh outputs retain archive hashes, without full member inventories. Each completed fresh pipeline was followed by a second cleanup outside the timed interval. Its ZIP identity flags do not establish general cleanup idempotence.

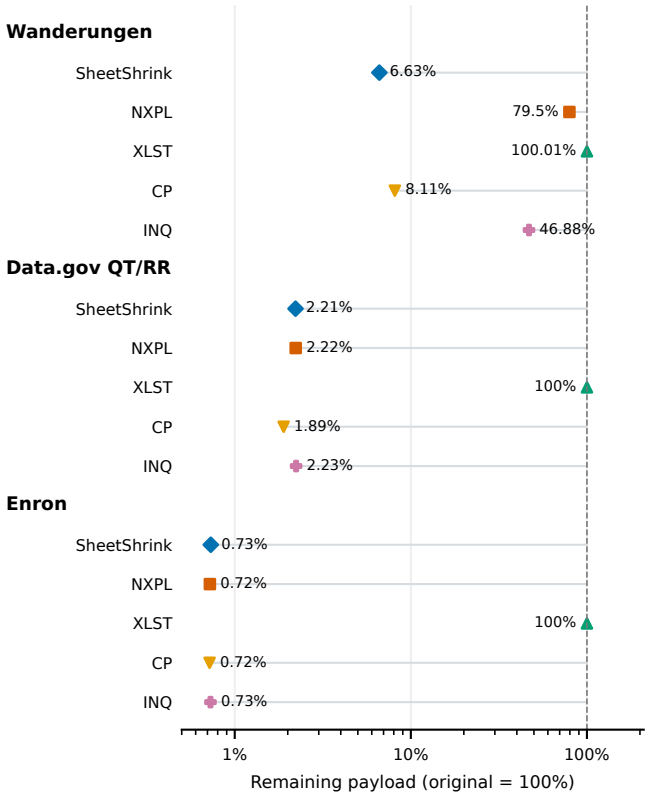


Fig. 10. Remaining uncompressed payload of the selected workbooks, relative to all ZIP-member bytes in the original. NXPL, XLST, CP and INQ denote NXPowerLite, XLStylesTool, Check Performance and Inquire. Lower is smaller; the x-axis is logarithmic. Original payloads are 248.39 MB (Wanderungen), 1,350.77 MB (QT/RR) and 353.15 MB (Enron). SheetShrink uses separately regenerated controls with verified native-candidate member identities. Tiny increases can round to 100%. Payload reduction and native visual preservation have separate evidence.

F. Enron selected-case follow-up

The Enron workbook contains 42,710,945 compressed bytes and 353,151,842 uncompressed member bytes. On the same larger-memory host, each of six variants received three openpyxl 3.1.5 open–save trials, with a 4 GiB process-tree commit limit, 8 GiB physical-RAM and commit reserve, and 300-second timeout [16]. Original and XLStylesTool reached the memory limit in every trial. Fresh SheetShrink cleanup plus open–save completed in a median 3.021 seconds from process creation to exit, with a maximum job-commit peak of 891.4 MiB. Prepared Check Performance, Inquire and NXPowerLite outputs completed in medians of 1.036, 1.054 and 1.101 seconds, respectively, with maximum job-commit peaks of approximately 535.6 MiB.

External-tool timings exclude preparation, while SheetShrink timings include fresh cleanup. They therefore cannot establish an end-to-end speed ranking. Memory-limited trials remain censored. Recorded commit peaks slightly exceeded 4 GiB and were retained unchanged. Concurrent native comparisons and retained filesystem caches qualify these results.

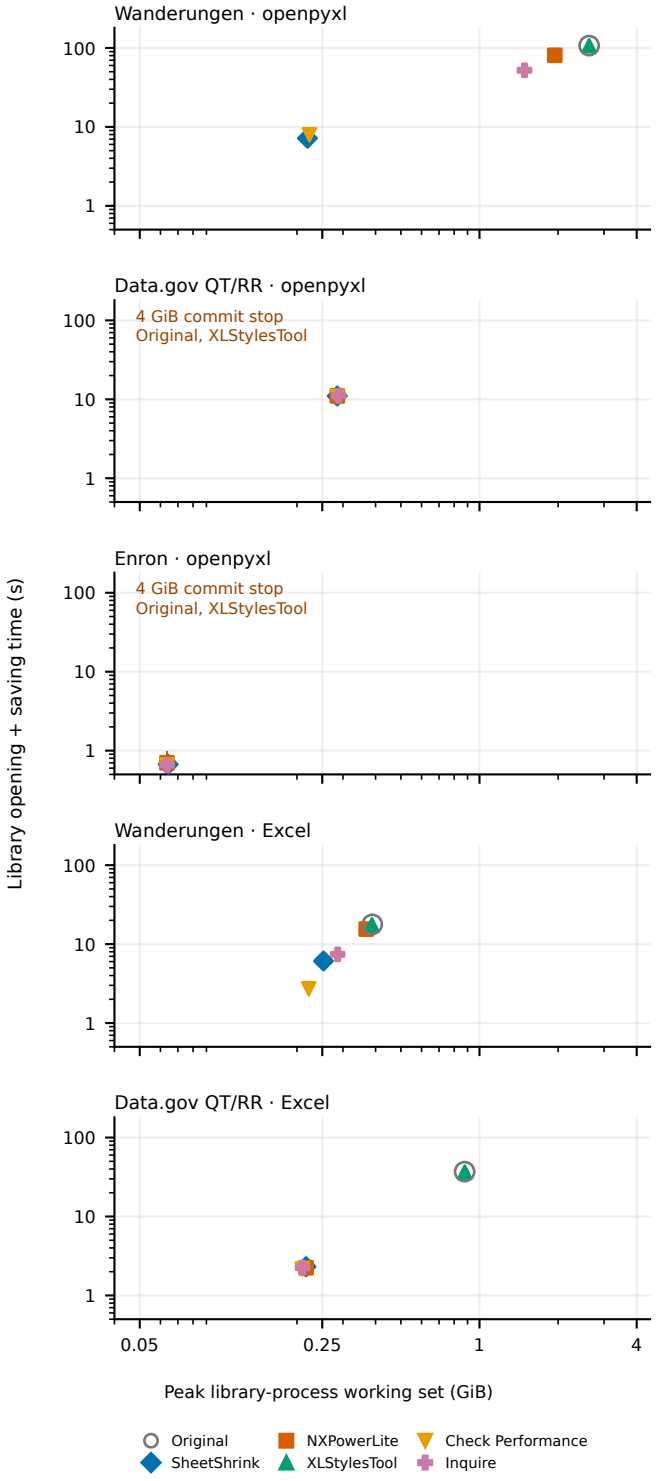


Fig. 11. Library open–save cost on selected workbooks, excluding cleanup. Points show median elapsed time and maximum library-process working set over three attempts; vertical bars show the elapsed-time range. Memory-stopped attempts have no completed-time point. SheetShrink uses fresh outputs; other tools use prepared outputs. Wanderungen and QT/RR use the laptop, Enron the larger-memory host. All jobs have a 4 GiB commit limit, distinct from working set. Comparisons are within panels.

After timing, a fresh SheetShrink control matched the native candidate's eighteen uncompressed member names, lengths and hashes. Its output contains 384,724 compressed bytes and 2,581,673 uncompressed member bytes. This check links the control to the visual audit within its selected coverage. Timed trials retain archive hashes, without full per-trial member inventories.

G. Distributed comparator measurements

We distribute native tool comparisons across Windows hosts because opening, cleanup and saving are costly. Each result records its host, input identity, configuration and timing boundary. Cross-tool ratios compare the same original workbook with SheetShrink on the same host. Each host/tool combination is plotted separately. Workload, tool behavior and session conditions still affect these ratios. The frozen selection comprises the 49 workbooks with the largest absolute compressed-file savings in each corpus's earlier SheetShrink screening. Results apply to these selected cases.

NXPowerLite runs on a Lenovo 81YQ with a Ryzen 5 4500U (six cores and six logical processors), 8 GiB installed RAM (7.37 GiB usable), a 512 GB Samsung MZALQ512HALU NVMe SSD and Windows 11 Home build 26200. The power scheme is Energy Star. Resource records include other open Excel and validation processes.

a) Comparator configuration: NXPowerLite Desktop 11.0.1 x64 uses a custom Excel profile with only excess cell-formatting cleanup enabled. All other cleanup options are disabled to preserve visible content. An output audit checks whether images remain unchanged.

We compare media, drawings, charts and embedded objects byte-for-byte against the originals. Of 147 workbooks, 129 became smaller and ten remained unchanged. Eight Enron inputs failed with an internal NXPowerLite error, including after restarting and retrying. Five outputs recompressed PNGs without changing decoded RGBA pixels or dimensions. Another changed an embedded Word document. These six outputs are retained separately and excluded from the formatting-focused timing figure. The remaining 133 outputs passed the audit. Worksheet appearance still requires separate visual validation.

XLStylesTool UWP 3.0.1.0 runs on the same host, with forced style normalization, name unhiding and wholesale external-name removal disabled. We retain 122 hash-matched inputs with confirmed primary conversions and complete timing and resource records. The other 25 inputs lack complete timing evidence and are excluded. All 122 outputs preserve audited media and object parts byte-for-byte. The joint figure uses the 110 inputs shared by both tools, comprising 21 Destatis, 41 Enron and 48 Data.gov workbooks, with the same repeated SheetShrink baselines. Table I reports NXPowerLite resources for all 133 audited outputs.

Check Performance uses Excel 16.0.20527.20150 on the Lenovo host, with a dedicated Excel worker on a private desktop for each input. Of the same 110 originals, 102 primary runs completed, one partially completed and seven failed, including one timeout. Completed runs comprise 86 Optimize All actions and sixteen cases requiring no optimization, covering 21 Destatis, 36 Enron and 45 Data.gov inputs. The partial run saved its output before failing to update progress. Its successful retry is counted separately. All original hashes remained unchanged.

Check Performance timing spans wrapper startup to exit, including control saves, optimization, artifact creation and shutdown. Input staging and independent hashing are excluded. Optimization and saving were timed together. Complete primary runs retain Excel CPU time through exit. Working set and private bytes were sampled at a median mean interval of 251 ms. All 990 reused Lenovo SheetShrink measurements list a concurrent Check Performance runner, whose activity is unknown. Median workflow ratios are 20.19, 22.75 and 4.84 for Destatis, Enron and Data.gov. Background-load uncertainty and different timing boundaries limit interpretation.

Inquire runs on an HP EliteBook 645 G11 with a Ryzen 5 PRO 7535U (six cores, twelve logical processors), 32 GiB RAM, a 512 GB SOLIDIGM NVMe SSD and Windows 11 Enterprise build 26200.9711. Excel and Inquire versions are 16.0.17932.21056 and 16.0.16626.20076. Of the same 110 originals, 90 primary runs completed, six partially completed and fourteen failed. Diagnostic retries are counted separately. Completed runs comprise 19 Destatis, 28 Enron and 43 Data.gov workbooks.

Each Inquire input has one historical timing covering opening, cleanup with internal saving, explicit saving and shutdown. Per-case records retain detailed timing boundaries. Partial and failed runs receive no runtime ratio. Hardware and version attribution partly relies on session records and later inventories. Of 90 complete pairs, 41 Inquire work directories used local NVMe, 48 used a USB-connected Samsung 870 QVO SSD, and one has unknown storage. All paired SheetShrink trials used local NVMe. Median workflow ratios are 19.36, 15.18 and 12.04 for Destatis, Enron and Data.gov. Different timing boundaries and storage conditions limit interpretation.

On the HP host, SheetShrink uses twelve Rayon and twelve writer workers with one active workbook. Each input receives one warm-up and seven measured trials, extended to eleven if the relative interquartile range exceeds 10%. All 110 inputs completed, totaling 110 warm-ups and 806 measured trials. Uncompressed output contents matched across repetitions. Trials use fresh Windows processes, warm filesystem caches, seeded shuffled ordering and a 1,800 s timeout. Timing spans process start to exit, excluding hashing and verification. The host retains its institutional power scheme and background workload. No comparator Excel process ran concurrently.

b) Timing and resources: NXPowerLite and XLStylesTool each have one primary timing per input. Their execution cost limits repetition, leaving timing variability unestimated. Diagnostic retries are counted separately. Timing includes GUI automation, output confirmation and initial package verification. CPU measurements sum process-tree user and kernel time. Working-set and private-byte peaks are sampled

at nominal 250 ms intervals. Each tool reuses its running application with one conversion worker and a 512 MiB RAM and commit reserve.

On the Lenovo host, SheetShrink uses an optimized release build, Default cleanup, one active workbook, one Rayon worker and one writer. One warm-up round precedes nine measured rounds over the same 147 inputs. Each round shuffles input order with a recorded seed. Each file runs in a fresh CLI process, timed through exit, excluding output hashing. Filesystem caches remain warm. We record direct-child CPU time and OS peak working set, sampling private memory every nominal 10 ms. These memory measurements differ from the GUI tools and are reported separately. All 1,470 runs completed. Each input produced identical package hashes across repetitions. All 147 final outputs preserved audited media and object bytes. The median relative within-input interquartile range, $(Q_3 - Q_1)/\tilde{t}$, was 0.84% across nine measured repetitions.

Figure 12 shows times and same-host tool/SheetShrink ratios for each workbook. Ratios divide the primary comparator time by the SheetShrink median from nine Lenovo trials or seven to eleven HP trials. Quartiles across workbooks describe differences within this selected stress set. They do not estimate repeated-trial uncertainty [17]. Ratios include interface and confirmation costs and compare tools with different cleanup policies. Opening time and cleanup-plus-opening time are measured separately in the opening experiment.

H. Native validation of preservation rules

a) Row-height preservation.: In the Traffic workbook, replacing repeated empty-cell styles with a row style reduced Excel's automatically calculated row height from 12.75 to 9 points. This shifted the content below the row and changed 3,408 pixels in a reference region specified independently of the capture selector. We therefore require an explicit custom row height before applying this transformation. With this requirement, the native captures were byte-identical [9].

b) Font representatives.: Two Destatis workbooks show why blank cells may need to retain font information. In a higher-education workbook, deleting 87 blank cells with explicit default-font and no-fill settings changed 25,640 pixels. The capture geometry was unchanged, and comparing the original with itself produced identical pixels. Keeping the required font representatives restored equality in an independently specified reference region. Legacy print-name aliases were normalized to open the workbook, with this preparation recorded separately.

In a cultural-finance workbook, a blank cell's non-default font affected automatic row height even without an `applyFont` flag. A stored height without `customHeight` still allowed Excel to change the height from 15 to 12.75 points. Keeping the representative restored matching pixels and geometry in both the independent reference region and all 350 regions selected by the targeted capture policy.

c) Background definitions.: In a Destatis horticulture workbook, SheetShrink reduced column definitions from 128,912 to 2,197. A corrected DeltaSieve background-envelope guard reduced the capture selection from 16,101 to 165 regions; all 165 resulting image pairs were pixel-identical.

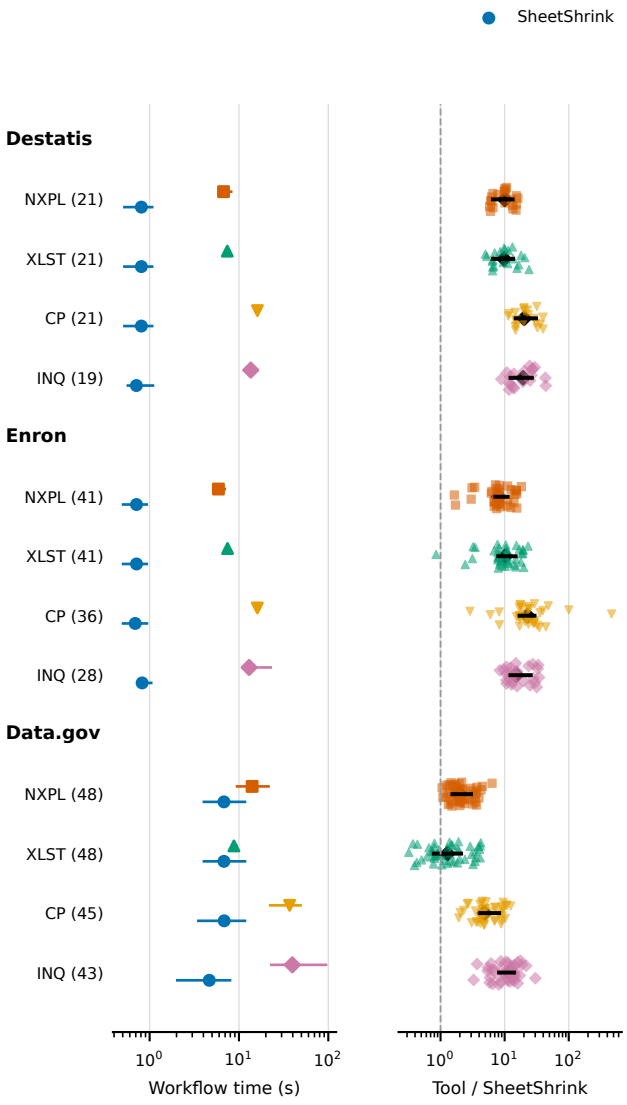


Fig. 12. Cleanup times and tool/SheetShrink ratios by corpus; parentheses give pair counts. Time summaries and black ratio marks show medians and interquartile ranges. NXPL: NXPowerLite; XLST: XLStylesTool; CP: Check Performance; INQ: Inquire.

d) Validation boundaries.: In Wanderungen, excluding an empty trailing context row removed a spurious height finding. All 37 captured image pairs were pixel-identical under the DeltaSieve capture policy, while separate effective-height observations on two other worksheets remained unresolved [18]. These case studies validate specific preservation conditions within the recorded capture scopes; they do not establish exhaustive workbook or corpus-wide visual equivalence.

TABLE I
MEDIAN RESOURCE OBSERVATIONS ACROSS PAIRED WORKBOOKS ON THE LAPTOP. CPU IS PROCESS TIME IN SECONDS. WS IS WORKING SET IN MIB, USING OS PEAK COUNTERS FOR SHEETSHRINK AND SAMPLED PEAKS FOR NXPOWERLITE.

Corpus	Pairs	CPU SheetShrink	CPU NXPowerLite	WS SheetShrink	WS NXPowerLite
Destatis	44	1.24	5.45	54	248
Enron	41	0.70	4.19	51	306
Data.gov	48	6.73	10.49	415	146

I. Targeted visual outcomes on the Top49 selection

A version-matched audit of the 49 workbooks with the largest absolute SheetShrink reductions in each corpus retained 42 Destatis, 46 Data.gov, and 46 Enron comparisons against the selected originals. Separately documented normalized baselines supplied five further Destatis and two further Enron observations. The remaining two, three, and one slots, respectively, had no eligible observation. Across both baseline categories, zero Destatis, seven Data.gov, and five Enron workbooks had recorded differences. These counts describe the selected capture scope and include findings whose cause remains unresolved. The five Enron workbooks had six chart findings of 2–26 changed pixels with equal image dimensions and no reported geometry change. The missing Enron workbook required repair when opened in Excel, including after a separate legacy reconversion.

J. Evidence boundary

The selected controls demonstrate concrete preservation conditions and recorded failures. They do not estimate corpus-wide defect prevalence. Reduction results cover the frozen XLSX inventories. Matched cleanup-plus-opening measurements and independent native references cover selected cases. Broader reference coverage and matched whole-corpus cost remain open. Detailed development attempts, input hashes, native images and producer versions remain in the source-linked artifact.

V. RELATED WORK

A. Workbook bloat and cleanup tools

Microsoft Check Performance removes cells associated with unnecessary formatting and other stored information from a loaded workbook [19]. Its documentation warns that optimization can remove pixel-art cells. Inquire's Clean Excess Cell Formatting command removes excess cells beyond the last nonblank cell and can change conditional-formatting coverage [20]. O'Beirne's published analysis of Excel 2013 Inquire discusses this command and criticizes its lack of information about planned and completed removals [21].

Package-level cleanup also predates SheetShrink. XLStylesTool's developer describes an Open-XML-based cleaner for excessive and corrupted styles that operates independently of Excel's object model [22]. NXPowerLite Desktop 10 lists excess XLSX cell-format cleanup among its features [23]. Its engine description discusses formatted blank cells as a source of large worksheet parts [24]. Their available descriptions do not establish the guarded removal rules, compact row-font witnesses or native preservation outcomes examined here. SheetShrink's positioning therefore includes its effect-aware rules, retained representatives and measured costs alongside its execution before native loading.

B. Spreadsheet scalability and system optimization

Mack et al.'s CHI Extended Abstracts case study characterizes scalability problems reported in spreadsheet forums [25]. Rahman et al. benchmark Excel, LibreOffice Calc and Google Sheets on spreadsheet computations and examine indexing, data layout, incremental evaluation and shared computation [26]. Their workloads assess system responsiveness and computation, while SheetShrink measures removable persisted representation and the cost of preparing workbooks for subsequent use.

DataSpread's storage engine integrates spreadsheet presentation with database storage and positional access [27]. Anti-Freeze schedules asynchronous formula computation to retain interactivity during updates [28]. Hillview distributes exploration of large datasets and uses compact visualization sketches [29]. TACO exploits regular formula-reference patterns to compact dependency graphs and query them directly [30]. SheetShrink transforms an existing XLSX package into another editable workbook under a stated preservation contract.

C. Spreadsheet loading and specialized parsing

SheetReader studies specialized XML parsing for efficient spreadsheet ingestion, including the coupling of decompression, parsing and parallel execution [31]. During baseline ingestion, we encountered an empty-cell parsing defect in SheetReader and submitted a correction with regression fixtures [32].

SheetReader loads worksheet data into analytical representations. SheetShrink rewrites an editable workbook whose required contents, dependencies and presentation must survive cleanup. Its byte-oriented scanner therefore combines XML recognition with preservation-specific guards. Corpus-scale processing and targeted regression controls exercise this implementation within its supported domain, while additional serializations and semantic cases can remain untested.

D. Spreadsheet refactoring and validation

Hermans et al. detect formula smells and suggest refactorings to improve spreadsheet maintainability [33]. SheetShrink's removal rules additionally account for formatting inheritance and native layout effects of stored cells.

Spreadsheet slicing integrates dependency reasoning with testing and debugging [34]. Dependencies also constrain removal when retained formulas or graphical objects refer to

candidate content. Workbook comparison tools provide observations about cell and format differences [35]. Our native validation protocol uses DeltaSieve's region selection and capture workflow [3], with independent references and mutation controls to evaluate selected coverage.

VI. Discussion and Limitations

The preservation contract is narrower than all possible spreadsheet behavior. Rendering depends on the installed Excel version, fonts and calculation state. A selected-region observation has limited scope. Unsupported XML and unsuccessful opening or saving remain visible outcomes. The independent reference sample estimates behavior on its eligible population and cannot establish a universal absence of regressions.

Large reductions can coexist with small preservation witnesses. Font metadata, inheritance masks and content-bearing hidden tails therefore require separate conditions. The constant-tail policy remains a structural heuristic. The fast CRC/length identity shortcut is collision-prone, and the selected verification mode must accompany measurements. The built-in-name predicate also lacks a formula-body equivalence check.

VII. Conclusion

SheetShrink addresses spreadsheet redundancy through effect-aware removal and compact preservation. Its implementation and evaluation distinguish compressed size, XML changes, execution cost and observed presentation.

Acknowledgment of AI Assistance

OpenAI ChatGPT and Codex assisted with drafting and editing the Abstract and Sections I–VII, literature discovery and organization, implementation and refactoring, test and experiment scripts, figure-generation code, and analysis of retained experiment evidence. This assistance included the method diagrams, XML examples and execution visualizations. Local agent workflows using these systems also supported overnight validation campaigns by inspecting status and logs, investigating stalled attempts, and documenting candidate visual regressions for author review.

The author directed development, reviewed source code, and performed manual testing and debugging. Agent observations supported investigation. Reported validation outcomes are grounded in the retained native captures and comparison records. The author remains responsible for the scientific claims, references, evidence selection, originality and final submitted content.

References

[1] Microsoft, "Row.CustomHeight property," Open XML SDK documentation, accessed November 9, 2026. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.spreadsheet.row.customheight

[2] SheetShrink project, "XLSX shrinking behavior," Versioned software specification, revision a7eef1a, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/a7eef1a/docs/behavior/shrinker.md

[3] "DeltaSieve: Targeted visual validation of spreadsheet transformations," Companion working manuscript prepared for ICDE 2027, 2026, supplementary material, related-manuscripts/deltasieve.pdf.

[4] DeltaSieve project, "Native wrapping and merged-cell reference controls," Versioned experiment receipts, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/ab016bd2/docs/visual-validation/experiments/independent-reference/layout-controls-20261111.json

[5] ——, "INDIRECT conditional-format fallback controls," Versioned experiment receipts, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/6faefcaa/docs/visual-validation/experiments/independent-reference/dependency-controls-20261111.json

[6] SheetShrink project, "Preserve attributed cell value elements," Revision 88c77f9, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/commit/88c77f9557cd63d6ed887c5389ebb7e4a55452fc

[7] ——, "Preserve defined-name worksheet scope," Revision 5822800, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/commit/582280043a56a0346cb891fb0b9bbf23b4a3a875

[8] ——, "Do not publish outputs from incomplete worksheet parses," Revision c608d4e, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/commit/c608d4e5155fcbf584f9e2154a378c6f5fa42e2b

[9] ——, "Dense row promotion and inherited height," Versioned protocol and evidence, revision e5e4fe8, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/e5e4fe8/docs/visual-validation/experiments/traffic-row-height/README.md

[10] ——, "Hidden constant-tail investigation and wanderungen controls," Versioned evidence, revision 1b9ee98, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/1b9ee98/docs/visual-validation/investigations/hidden-constant-tails-20261104.md

[11] ——, "Xlsx shrinking behavior," Versioned protocol and evidence, revision 7ca7c2a, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/7ca7c2a/docs/behavior/shrinker.md

[12] ——, "Complete-corpus distribution and replay measurements," Versioned evidence, revision ce3722cb, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/ce3722cb/docs/visual-validation/experiments/full-corpus-distribution-20261111.json

[13] ——, "Prefixed worksheet data inventory," Inventory, normalization trials and sampled native controls, revision 44d73f3, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/44d73f3/docs/visual-validation/experiments/prefixed-data/README.md

[14] ——, "Confirmed default visual regressions," Controlled restoration evidence, revision 1b9ee98, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/1b9ee98/docs/visual-validation/confirmed-default-regressions.md

[15] "Bounded opening and saving follow-up," Supplementary experimental artifact, 2026, protocol, individual receipts, source snapshot and candidate-identity controls in evidence/bounded-open-save-followup.

[16] "Enron selected-case opening and saving follow-up," Supplementary experimental artifact, 2026, eighteen bounded receipts, protocol and representative native candidate identity control in evidence/enron-open-save-followup.

[17] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 2015, pp. 73:1–73:12.

[18] DeltaSieve project, "Targeted visual validation behavior, empty trailing row exclusion," Versioned protocol and evidence, revision 7ca7c2a, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/7ca7c2a/docs/behavior/targeted-validation.md

[19] Microsoft, "Cleanup cells in your workbook," accessed November 9, 2026. [Online]. Available: https://support.microsoft.com/en-us/excel/cleanup-cells-in-your-workbook

[20] ——, "Clean excess cell formatting on a worksheet in Excel for Windows," accessed November 9, 2026. [Online]. Available: https://support.microsoft.com/en-us/excel/clean-excess-cell-formatting-on-a-worksheet-in-excel-for-windows

[21] P. O'Beirne, "Excel 2013 Spreadsheet Inquire," in *Proceedings of the EuSpRIG 2013 Conference "Spreadsheet Risk Management"*. European Spreadsheet Risks Interest Group, 2013, also available as arXiv:1401.7586, deposited in 2014. [Online]. Available: https://eusprig.org/wp-content/uploads/1401.7586.pdf

[22] XLGeek, ".NET4.0 version of the XLStylesTool is now available," 2011, developer description, accessed November 9, 2026. [Online]. Available: https://sergeig888.wordpress.com/2011/03/21/net4-0-version-of-the-xlstylestool-is-now-available/

[23] M. Power, "What's new in NXPowerLite Desktop 10," vendor release description, accessed November 9, 2026. [Online]. Available: https://neuxpower.com/blog/whats-new-in-nxpowerlite-desktop-10

[24] ——, "Compression engine major upgrade—version 10," vendor description, accessed November 9, 2026. [Online]. Available: https://neuxpower.com/blog/wecompress-file-compressor-engine-upgrade

[25] K. Mack, J. Lee, K. Chang, K. Karahalios, and A. Parameswaran, "Characterizing scalability issues in spreadsheet software using online forums," in *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems*. ACM, 2018. [Online]. Available: https://arxiv.org/abs/1801.03829

[26] S. Rahman, K. Mack, M. Bendre, R. Zhang, K. Karahalios, and A. Parameswaran, "Benchmarking spreadsheet systems," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, 2020, pp. 1589–1599. [Online]. Available: https://sajjadur.net/files/benchmarking_spreadsheets.pdf

[27] M. Bendre, V. Venkataraman, X. Zhou, K. C.-C. Chang, and A. Parameswaran, "Towards a holistic integration of spreadsheets with databases: A scalable storage engine for presentational data management," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 113–124. [Online]. Available: https://arxiv.org/abs/1708.06712

[28] M. Bendre, T. Wattanawaroon, K. Mack, K. Chang, and A. Parameswaran, "Anti-Freeze for large and complex spreadsheets: Asynchronous formula computation," in *Proceedings of the 2019 International Conference on Management of Data*. ACM, 2019, pp. 1277–1294. [Online]. Available: https://people.eecs.berkeley.edu/~adityagp/papers/dataspread-async.pdf

[29] M. Budiu, P. Gopalan, L. Suresh, U. Wieder, H. Kruiger, and M. K. Aguilera, "Hillview: A trillion-cell spreadsheet for big data," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1442–1457, 2019. [Online]. Available: https://arxiv.org/abs/1907.04827

[30] D. Tang, F. Chen, C. De Leon, T. Wattanawaroon, J. Yun, S. Seshadri, and A. G. Parameswaran, "Efficient and compact spreadsheet formula graphs," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 2634–2646. [Online]. Available: https://arxiv.org/abs/2302.05482

[31] H. Gavriilidis, F. Henze, E. T. Zacharatou, and V. Markl, "SheetReader: Efficient specialized spreadsheet parsing," *Information Systems*, 2023.

[32] A. Aue-Johr, "Fix empty-cell parsing and preserve following cell positions," SheetReader core, pull request 5, submitted November 11, 2026. [Online]. Available: https://github.com/polydbms/sheetreader-core/pull/5

[33] F. Hermans, M. Pinzger, and A. van Deursen, "Detecting and refactoring code smells in spreadsheet formulas," *Empirical Software Engineering*, vol. 20, no. 2, pp. 549–575, 2015.

[34] J. Reichwein, G. Rothermel, and M. Burnett, "Slicing spreadsheets: An integrated methodology for spreadsheet testing and debugging," in *Proceedings of the Second Conference on Domain-Specific Languages*. USENIX, 1999, pp. 25–38. [Online]. Available: https://www.usenix.org/legacy/publications/library/proceedings/dsl99/full_papers/reichwein/reichwein_html/index.html

[35] Microsoft, "Basic tasks in spreadsheet compare," Microsoft Support, accessed November 8, 2026. [Online]. Available: https://support.microsoft.com/en-gb/excel/basic-tasks-in-spreadsheet-compare